

TECHBIZ SECURITY

INTERNSHIP REPORT

Week 2 - Task 6 (SOC)

Topic: Linux Basics and Log File Analysis

Submitted By: MANOJ

Role: Cyber Security Intern

Internship Organisation: TechBiz Security

Internship ID: TBS-INT-2026-518

Submitted Date: 31 August 2026

Table of Contents

1. Introduction
2. Objectives
3. Tools and Environment
4. Linux Log Directory
5. Apache Log Files
6. Apache Service Verification
7. Web Server Request Testing
8. Apache Access Log Analysis
9. Normal HTTP Request Logging
10. HTTP 404 Error Analysis
11. Log Monitoring with Tail
12. TryHackMe - Investigating Windows
13. TryHackMe - Intro to Logs
14. SOC Relevance
15. Key Learnings
16. Security Observations
17. Challenges and Troubleshooting
18. Outcome
19. Conclusion
20. Evidence / Screenshot Index

1. Introduction

This report documents the work carried out during Week 2, Task 6 of the Security Operations Center (SOC) internship at TechBiz Security. The task focused on building a practical understanding of Linux fundamentals and Linux log files, with particular emphasis on Apache web-server logging. The activities covered navigating the Linux file system, locating and examining log directories, verifying and troubleshooting the Apache2 service, generating web traffic using curl, and analysing the resulting access-log entries, including both successful requests and HTTP 404 errors.

In addition to the hands-on Linux and Apache exercises, the task also included two TryHackMe learning rooms, "Investigating Windows" and "Intro to Logs," both of which were completed at 100 percent. These rooms reinforced the practical skills demonstrated in the Linux environment by introducing Windows-based investigation techniques and the broader theoretical foundations of logging that are used across SOC operations.

This report presents the objectives, tools, methodology, and observations of the task, supported by screenshot evidence collected during the exercise.

2. Objectives

- Understand basic Linux commands used during SOC activities.
- Navigate Linux directories and log locations.
- Understand the purpose of the /var/log directory.
- Identify and examine Apache web-server logs.
- Understand the structure and fields of access-log entries.
- Observe HTTP requests and response status codes.
- Identify unsuccessful requests such as HTTP 404 responses.
- Understand how logs can be used as evidence during security monitoring and investigation.
- Practice log-analysis concepts through TryHackMe SOC-related learning rooms.

3. Tools and Environment

The following tools and environment were used to complete this task.

Component	Details
Operating System	Kali Linux
Web Server	Apache2
Log Directory	/var/log
Apache Log Directory	/var/log/apache2/
Command-Line Tool	curl
Log Viewing / Monitoring Commands	tail
Training Platform	TryHackMe

4. Linux Log Directory

The /var/log directory is the standard location on a Linux system where log files generated by the operating system and various installed services are stored. Reviewing the contents of this directory (see the terminal output in the screenshot below) shows that multiple services maintain their own log files or subdirectories, including apache2, mysql, nginx, samba, sysstat, and several other service-specific entries such as apt, journal, lightdm, and runit.

Each subdirectory or file corresponds to a specific service and stores information relevant to that service's activity, errors, and events. For a SOC analyst, knowing where these logs are located is a fundamental skill, since most investigations begin with identifying which log source is relevant to the event under review. Without a clear understanding of the /var/log structure, an analyst would not be able to quickly locate the evidence needed to investigate a security incident.

[Insert Screenshot: 01_apache_access_log.png]

```
manuara@kali:~$ ls /var/log
alternatives.log  apache2  boot.log  btmap  dpkg.log  fontconfig.log  inetd  journal  lightdm  macchanger.log.1.gz  mysql  notus-scanner  README  runit  speech-dispatcher  sysstat  Xorg.0.log  Xorg.0.log.old
alternatives.log.1  apt      boot.log.1  btmap.1  dpkg.log.1  gvm      installer  lastlog  macchanger.log  mosquitto  nginx  openvpn  private  redis  samba  stunnel  wtmp

manuara@kali:~$ sudo ls -ls /var/log/apache2/
total 8.0K
-rw-r----- 1 root adm  0 May 25 10:54 access.log
-rw-r----- 1 root adm 1.1K Sep  3 2025 access.log.1
-rw-r----- 1 root adm  0 May 25 10:54 error.log
-rw-r----- 1 root adm 1.0K Sep  3 2025 error.log.1
-rw-r----- 1 root adm  0 Jun 15 2025 other_vhosts_access.log

manuara@kali:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: inactive (dead)
     Docs: https://httpd.apache.org/docs/2.4/

manuara@kali:~$ sudo systemctl start apache2

manuara@kali:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Mon 2026-08-31 15:31:47 IST; 6s ago
     Invocation: da8415bc388427aa59d4b8731d34710
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 3142 (apache2)
   Status: "Processing requests ..."
     Tasks: 6 (limit: 65536)
    Memory: 21.6M (peak: 21.6M)
       CPU: 242ms
   CGroup: /system.slice/apache2.service
           └─3142 /usr/sbin/apache2 -k start -DFOREGROUND
             └─3148 /usr/sbin/apache2 -k start -DFOREGROUND
               └─3159 /usr/sbin/apache2 -k start -DFOREGROUND
                 └─3160 /usr/sbin/apache2 -k start -DFOREGROUND
                   └─3163 /usr/sbin/apache2 -k start -DFOREGROUND

Aug 31 15:31:46 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Aug 31 15:31:47 kali apache2[3142]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1. Set the 'ServerName' directive globally to suppress this message
Aug 31 15:31:47 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
```

Contents of /var/log and Apache2 service status verification

5. Apache Log Files

Within the /var/log/apache2/ directory, the following log files were observed:

File	Purpose
access.log	Current access log recording incoming web requests
access.log.1	Rotated (previous) access log
error.log	Current error log recording server-side errors
error.log.1	Rotated (previous) error log
other_vhosts_access.log	Access log for other configured virtual hosts

Access logs record every web request received by the Apache server, including the requesting IP address, the requested resource, the HTTP method, and the response status code. Error logs, in contrast, contain information about server-side errors and operational problems encountered by Apache while processing requests. During this task, the error.log file was not found to contain any

specific errors relevant to the exercises performed; the focus of the analysis was primarily on the access log.

6. Apache Service Verification

Before any HTTP requests could be tested, the status of the Apache2 service was checked using `systemctl`. The output showed that the service was initially inactive (dead). The service was then started using the `systemctl start apache2` command.

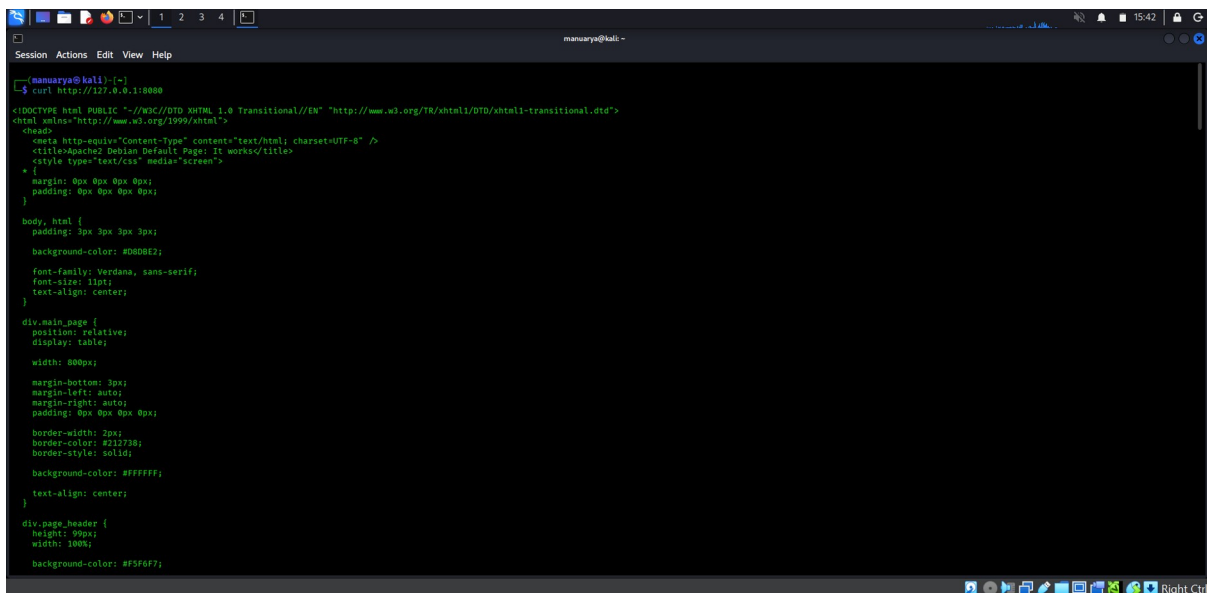
A follow-up status check confirmed that Apache2 was active (running) and processing requests, with the service log entries showing that the server had started successfully and was listening for connections. Verifying service status in this manner is a useful troubleshooting step, and it is also a relevant SOC skill, since analysts often need to confirm whether a service is running as part of an investigation or an incident response action.

7. Web Server Request Testing

HTTP requests were tested against the local Apache installation using `curl`. The Apache service in this environment was configured to listen on port 8080 rather than the default port 80. As a result, an initial request made to 127.0.0.1 without specifying the port did not succeed, since nothing was listening on port 80.

A subsequent request made to `http://127.0.0.1:8080` returned the Apache2 Debian default page successfully, confirming that the web server was reachable and functioning correctly once the correct port was used.

[Insert Screenshot: 02_apache_normal_request_log.png]



```
manuara@kali: ~
$ curl http://127.0.0.1:8080
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Debian Default Page: It works</title>
    <style type="text/css" media="screen">
      * {
        margin: 0px 0px 0px 0px;
        padding: 0px 0px 0px 0px;
      }
      body, html {
        padding: 3px 3px 3px 3px;
        background-color: #D0D0E2;
        font-family: Verdana, sans-serif;
        font-size: 11pt;
        text-align: center;
      }
      div.main_page {
        position: relative;
        display: table;
        width: 800px;
        margin-bottom: 3px;
        margin-left: auto;
        margin-right: auto;
        padding: 0px 0px 0px 0px;
        border-width: 2px;
        border-color: #333333;
        border-style: solid;
        background-color: #FFFFFF;
        text-align: center;
      }
      div.page_header {
        height: 90px;
        width: 100%;
        background-color: #F5F6F7;

```

curl request to `http://127.0.0.1:8080` returning the Apache default page

8. Apache Access Log Analysis

After the successful `curl` request to `http://127.0.0.1:8080`, a corresponding entry was recorded in the Apache access log. The relevant fields observed in this log entry are described below.

Field	Description
-------	-------------

Source IP Address	127.0.0.1 (the local host making the request)
Timestamp	Date and time the request was received, including timezone offset
HTTP Method	GET
Requested Resource	/ (root path)
HTTP Protocol Version	HTTP/1.1
HTTP Response Status Code	200
Response Size	10958 bytes
User-Agent	curl/8.20.0

The HTTP 200 status code indicates that the request was processed successfully and the requested resource was returned without error. This type of log entry represents normal, expected web-server activity. By reviewing access-log entries such as this one, a SOC analyst can build a baseline understanding of what typical, non-malicious traffic looks like on a monitored web server.

9. Normal HTTP Request Logging

This section demonstrates how a normal, legitimate HTTP request appears in the Apache access log immediately after it is generated. Using the tail command against `/var/log/apache2/access.log`, the request made to the root path (/) was confirmed to have been recorded with the expected fields: timestamp, source IP address, HTTP method, requested resource, protocol version, status code, and User-Agent.

This kind of correlation, where an observed action (the curl request) is matched to its corresponding log entry, is a foundational SOC skill. Timestamps allow analysts to establish a timeline of events, source IP addresses help identify who or what generated the traffic, requested URLs show what resource was targeted, HTTP methods indicate the type of action performed, status codes reveal the outcome of the request, and the User-Agent field can provide clues about the client or tool used to make the request.

10. HTTP 404 Error Analysis

To observe how the Apache access log records unsuccessful requests, curl was used to request non-existent resources, specifically `/nonexistent` and `/admin`. In both cases, the server returned an HTTP 404 Not Found response.

An HTTP 404 status code indicates that the requested resource could not be found on the server. Both requests were recorded in the access log with their respective timestamps, request paths, and the 404 status code, along with a response size of 474 bytes and the curl/8.20.0 User-Agent.

[Insert Screenshot: 03_apache_404_log_analysis.png]

```

manuara@kali:~$ cat /dev/null > /var/log/apache2/access.log
manuara@kali:~$ sudo tail -n 5 /var/log/apache2/access.log
127.0.0.1 - [31/Aug/2026:15:42:33 +0530] "GET / HTTP/1.1" 200 10950 "-" "curl/8.20.0"
manuara@kali:~$ curl http://127.0.0.1:8080/nonexistent
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>404 Not Found</title>
</head><body>
<p>The requested URL was not found on this server.</p>
</body></html>
manuara@kali:~$ curl http://127.0.0.1:8080/admin
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html><head>
<title>404 Not Found</title>
</head><body>
<p>The requested URL was not found on this server.</p>
</body></html>
manuara@kali:~$ sudo tail -n 10 /var/log/apache2/access.log
127.0.0.1 - [31/Aug/2026:15:42:33 +0530] "GET / HTTP/1.1" 200 10950 "-" "curl/8.20.0"
127.0.0.1 - [31/Aug/2026:15:43:48 +0530] "GET /nonexistent HTTP/1.1" 404 274 "-" "curl/8.20.0"
127.0.0.1 - [31/Aug/2026:15:45:35 +0530] "GET /admin HTTP/1.1" 404 474 "-" "curl/8.20.0"

```

HTTP 404 responses for /nonexistent and /admin, and corresponding access-log entries

From a SOC perspective, repeated requests for non-existent or sensitive-looking paths, such as /admin, can be worth further attention, since this pattern of behaviour is sometimes associated with reconnaissance or path-enumeration activity. However, a request such as /admin may equally represent legitimate testing or ordinary user activity. A single 404 response, on its own, does not prove that malicious activity has occurred; it becomes significant only when considered alongside additional context, such as request frequency, source IP reputation, or a broader pattern of similar requests.

11. Log Monitoring with Tail

The tail command was used to view recent entries in the Apache access log, which is a straightforward way to check the most recent web-server activity without opening the entire log file.

```
sudo tail -f /var/log/apache2/access.log
```

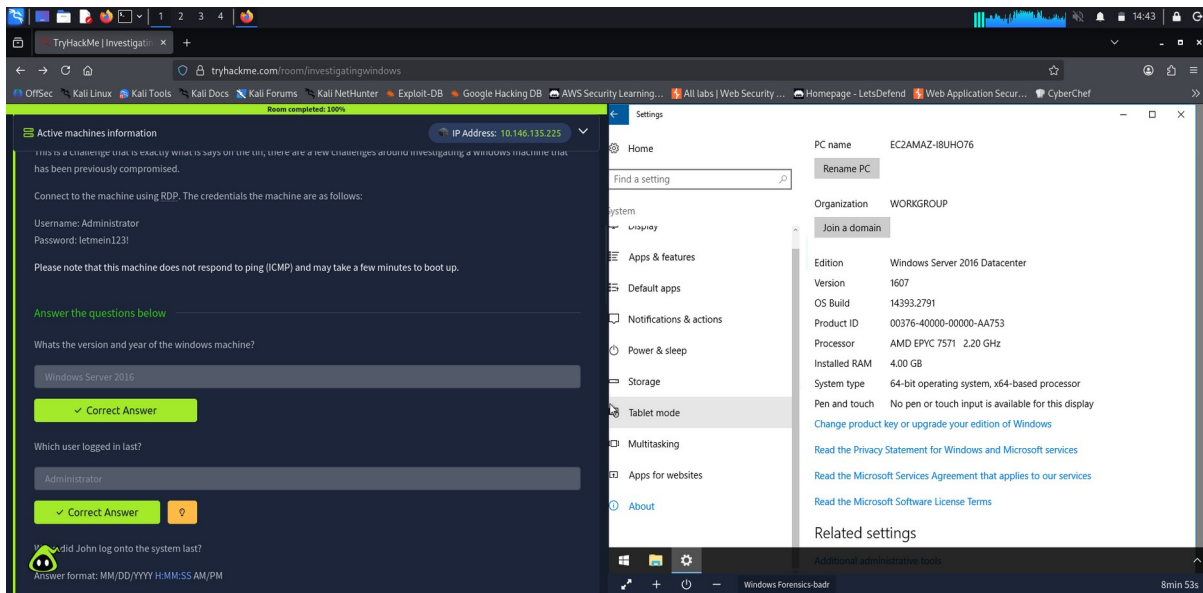
The -f flag allows tail to continuously follow the log file and display new entries as they are written, which supports real-time monitoring of web-server activity. This capability is important for SOC analysts, since it allows suspicious or unexpected activity to be observed as it happens rather than only during a retrospective review.

12. TryHackMe - Investigating Windows

The TryHackMe room "Investigating Windows" was completed at 100 percent as part of this task. The room focuses on investigating a Windows machine that has been compromised, requiring the identification of evidence to determine what actions may have been taken on the system.

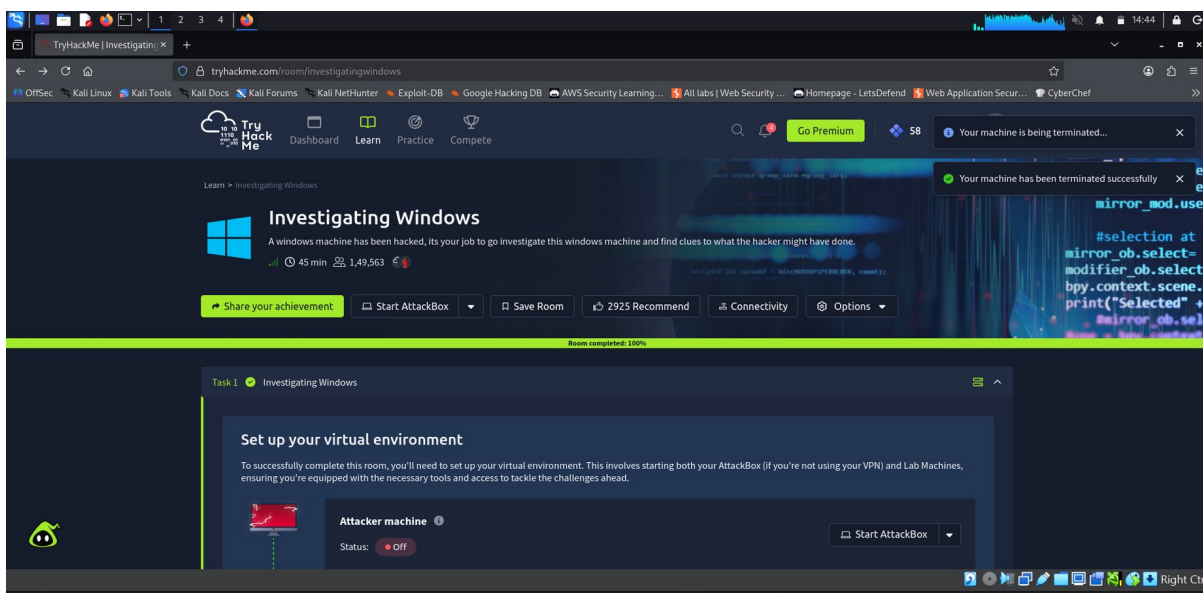
The investigation environment used a Windows Server 2016 Datacenter machine (Version 1607, OS Build 14393.2791), accessed with the Administrator account. Working through this room reinforced several practical investigation concepts relevant to SOC analysts, including identifying core system information, examining user account activity, and reviewing evidence from a system that had already been compromised in order to reconstruct what may have occurred.

[Insert Screenshot: 04_TryHackMe_Investigating_Windows_Task_Completion (1).png]



Investigating Windows - system information and question responses

[Insert Screenshot: 05_TryHackMe_Investigating_Windows__Completed.png]



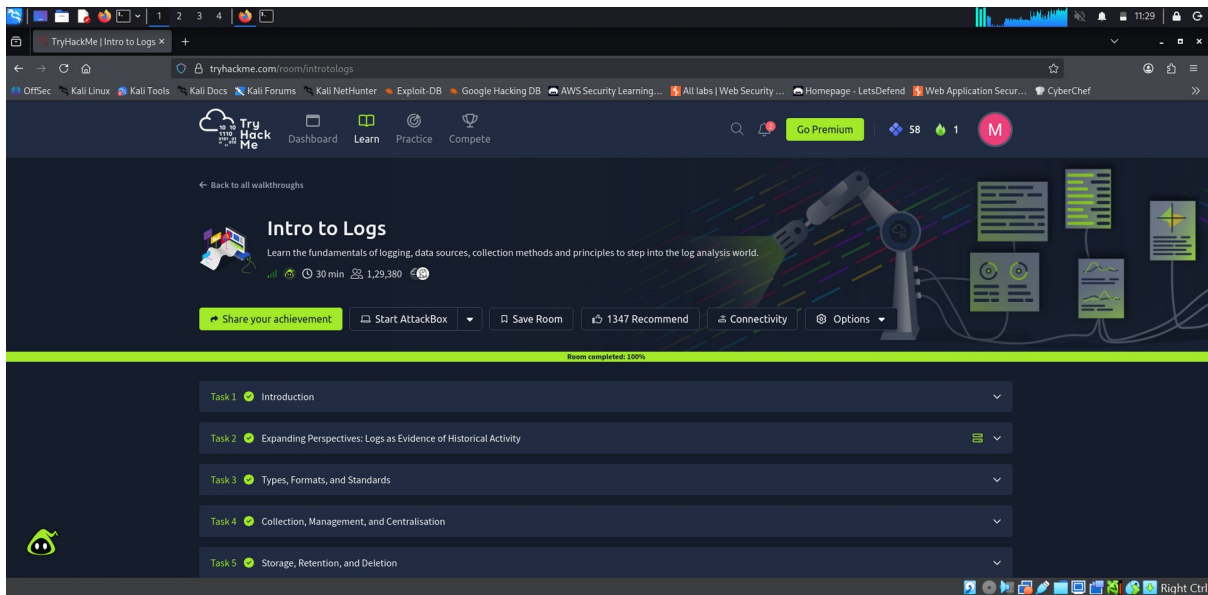
Investigating Windows room completed at 100 percent

13. TryHackMe - Intro to Logs

The TryHackMe room "Intro to Logs" was also completed at 100 percent. This room covers the fundamentals of logging, including data sources, the role of logs as evidence of historical activity, log types, formats, and standards, and the processes involved in log collection, management, centralisation, storage, retention, and deletion.

These foundational concepts directly relate to SOC operations, as they explain how logs are generated, gathered, and preserved across an organisation, and why proper log handling practices are essential for effective security monitoring and incident investigation.

[Insert Screenshot: 06_TryHackMe_Intro_to_Logs_Completed.png]



Intro to Logs room completed at 100 percent

14. SOC Relevance

The activities completed in this task connect directly to core functions performed within a Security Operations Center. Log collection and monitoring form the foundation of visibility into system and network activity, allowing analysts to identify events as they occur. Event identification and correlation, such as matching a curl request to its corresponding access-log entry, demonstrate how individual pieces of evidence are pieced together to understand a sequence of activity.

HTTP status-code analysis and web-server monitoring, as practiced with the Apache access log, are common tasks when investigating web-based activity or potential reconnaissance attempts. The Windows investigation exercise introduced additional evidence-analysis and forensic-investigation concepts, while the Intro to Logs room reinforced the broader principles of log management that support detection of abnormal activity, incident investigation, and security event triage across an organisation's systems.

15. Key Learnings

- Navigating the Linux /var/log directory and identifying service-specific log locations.
- Analysing Apache access-log entries and understanding their component fields.
- Understanding common HTTP status codes, including 200 and 404.
- Using curl to generate and test HTTP requests.
- Using tail, including tail -f, for log review and real-time monitoring.
- Identifying and interpreting HTTP 404 responses in an access log.
- Recognising the importance of timestamps and source IP addresses during investigation.
- Building foundational Windows investigation skills through a compromised-machine exercise.
- Understanding log management fundamentals, including collection, centralisation, storage, and retention.
- Developing a SOC investigation mindset when reviewing log evidence.

16. Security Observations

- Successful HTTP requests generated corresponding access-log entries with an HTTP 200 status code.
- Requests to invalid or non-existent paths generated HTTP 404 responses, which were correctly recorded in the access log.
- Apache web-server logs can provide useful evidence for monitoring web-server activity and supporting investigations.
- Repeated requests to sensitive or unusual paths, such as /admin, may warrant further investigation depending on surrounding context.
- A single HTTP 404 request should not automatically be treated as malicious activity.
- Proper log retention and centralisation practices improve an organisation's ability to investigate events effectively.

17. Challenges and Troubleshooting

- The initial curl request to 127.0.0.1 on port 80 failed, since Apache was configured to listen on port 8080 rather than the default port.
- The Apache2 service status was checked and found to be inactive.
- The Apache2 service was started successfully using systemctl.
- The correct port (8080) was then used for subsequent testing, which resolved the initial connection issue.
- Both successful and unsuccessful HTTP requests were analysed through their corresponding Apache access-log entries.

18. Outcome

Week 2, Task 6 improved practical understanding of Linux logging, Apache web-server monitoring, HTTP request analysis, Windows investigation, and general log-analysis fundamentals. Both TryHackMe rooms referenced in this report, Investigating Windows and Intro to Logs, were completed at 100 percent.

19. Conclusion

Understanding both Linux and Windows logs is an important foundation for Security Operations Center work. The practical Apache logging exercise carried out in this task demonstrated how user-generated requests produce corresponding log entries, and how HTTP status codes such as 200 and 404 can be analysed to distinguish normal activity from unsuccessful requests. Verifying and troubleshooting the Apache2 service also reinforced basic Linux service-management skills relevant to day-to-day SOC operations.

The TryHackMe Investigating Windows and Intro to Logs rooms further strengthened this foundation by introducing Windows-based investigation techniques and the broader theoretical principles behind log types, collection, management, and retention. Overall, this task provided practical exposure to identifying, interpreting, and using logs as evidence during security monitoring and investigation, which are core skills for any SOC analyst.

20. Evidence / Screenshot Index

No.	Screenshot Filename	Description
1	01_apache_access_log.png	Apache access log showing a successful HTTP request
2	02_apache_normal_request_log.png	Normal HTTP request and corresponding Apache log entry
3	03_apache_404_log_analysis.png	HTTP 404 responses and Apache access-log analysis
4	04_TryHackMe_Investigating_Windows_Task_Completion (1).png	TryHackMe Investigating Windows task completion evidence
5	05_TryHackMe_Investigating_Windows__Completed.png	TryHackMe Investigating Windows room completed at 100%
6	06_TryHackMe_Intro_to_Logs_Completed.png	TryHackMe Intro to Logs room completed at 100%